inversions - pair of elements that are out of order w.r.t sorting

1	4	3	2

(4, 3)
(4, 2)
(3, 2)

If array is in reverse sorted,

$$\#\text{inversions} = (n-1) + (n-2) + \dots + 3 + 2 + 1$$
$$= \frac{n(n-1)}{2}$$
$$= \Theta(n^2)$$

1	4	3	2

1	4	3	2

(4, 3)

1	3	4	2

(4, 2)
(3, 2)

1	2	3	4

In one particular iteration
$\#\text{swaps} = k$, $\#\text{comparisons} = k+1$

For all iteration (n-1 iterations)
$$\#\text{total comparisons} = \#\text{total swaps} + (n-1)$$

Total work = #swaps + #comps
$$= \#\text{swaps} + [\#\text{swaps} + (n-1)]$$
$$= 2\#\text{swaps} + (n-1)$$

$$\boxed{\text{Total work} = 2(\#\text{inversions}) + (n-1)}$$

WC: $\#\text{inversion} = \Theta(n^2)$
$\Rightarrow$ Total work $= \Theta(n^2)$

Partially sorted array (PSA): Array is partially sorted if $\#\text{inversions} = O(n)$

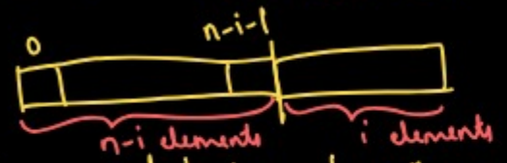
RT of IS for PSA $= \underbrace{2 \times O(n)} + (n-1) = \Theta(n)$

n	swaps	comp
$i=1$	k_1	k_1+1
$i=2$	k_2	k_2+1
$i=3$	k_3	k_3+1
$\vdots$		
$i=n-1$	k_{n-1}	$k_{n-1}+1$

$\#\text{total swaps} = k_1 + k_2 + \dots + k_{n-1}$

$\#\text{total comps} = (k_1 + k_2 + \dots + k_{n-1}) + (n-1)$

At start of iteration i, # sorted elements at the end = i



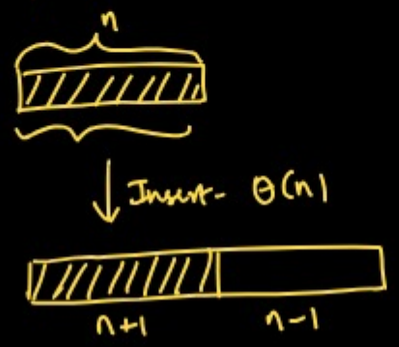
So pass needs to run only on the first 'n-i' elements

i=0

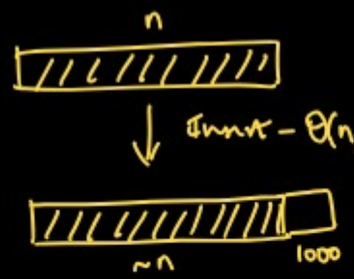
	0	1	2	3	4
	4	3	5	6	2
j=0	3	4	5	6	2
j=1	3	4	5	6	2
j=2	3	4	5	6	2
j=3	3	4	5	2	6

Dynamic arrays:

- If we have $\Theta(n)$ ops each of which take $\Theta(1)$ between two $\Theta(n)$ ops, then amortized running time of that op is $\Theta(1)$



We have to insert 'n-1' more elements (that each take $\Theta(1)$) before another



insert ops = 1000 before resize

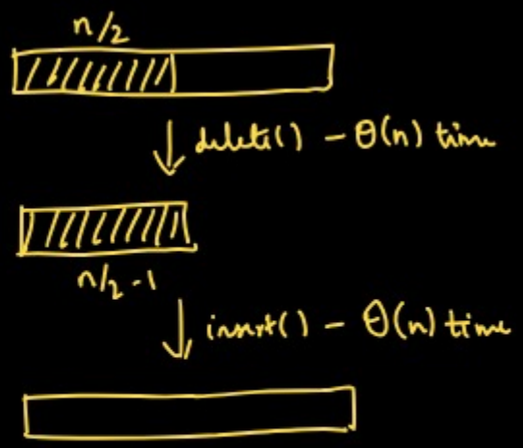
$\frac{n}{n}$ time

resize() $\Theta(n)$

Total RT = $1000 \times \Theta(1) + \Theta(n) = \Theta(n)$

Total # ops = 1000

Amortized RT for each op $= \frac{\Theta(n)}{1000} = \frac{kn}{1000} = \Theta(n)$



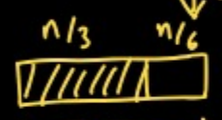
n' inserts, resize

$\underbrace{n \times \Theta(1)}_{n' \text{ inserts}} + \underbrace{\Theta(n)}_{\text{resize}} = \Theta(n)$

$n = \Theta(n)$

$\frac{\Theta(n)}{\Theta(n)} = \Theta(1)$

↓ delete() – $\Theta(n)$



↓ # inserts before doubling $= n/6 = \Theta(n)$

↓

$\Theta(1)$ – constant

insert() – $\Theta(n)$ – resize

$n/6$

D, I, I, ..., I, I

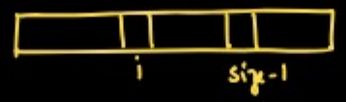
$\Theta(n)$ $\Theta(1)$ $\Theta(1)$ $\Theta(1)$ $\Theta(n)$

$n/6 = \Theta(n)$

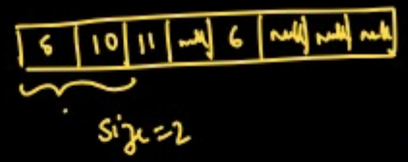
D, I, D, I ....

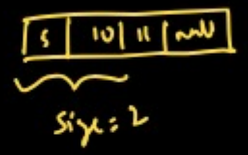
$\Theta(n)$ $\Theta(n)$ $\Theta(n)$ $\Theta(n)$

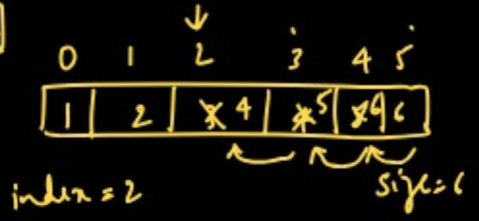
insert at index "i"



Every element from index 'i' to index 'size-1' should be moved one step to the right (from right to left)







index = 2

j = 3 index + 1

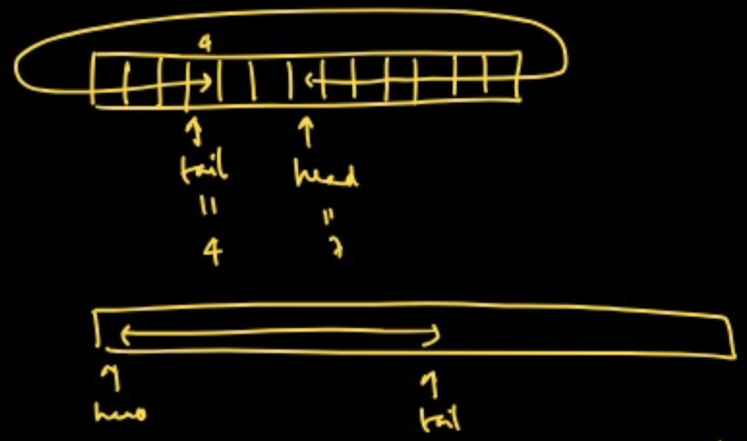
j = 4

j = 5 size - 1

enqueue - insert first

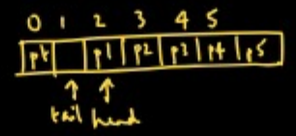
dequeue - delete last

$\Theta(1)$ for DLL



Queues using linked list

	SLL	SLL with tail	DLL
Option 1:			
· enqueue() - insertFirst()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
· dequeue() - deleteLast()	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Option 2:			
· enqueue() - insertLast()	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
· dequeue() - deleteFirst()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$



dividend - number that we are dividing

divisor - number with which we're dividing

$14/3$

$q = 4 \rightarrow$ quotient

$r = 2$

$-4 \% 3 = 2$
$-3 \% 3 = 0$
$-2 \% 3 = 1$
$-1 \% 3 = 2$
$0 \% 3 = 0$
$1 \% 3 = 1$
$2 \% 3 = 2$
$3 \% 3 = 0$
$4 \% 3 = 1$
$5 \% 3 = 2$

+3 +3 +3

Division Theorem

$m = n(q) + r$

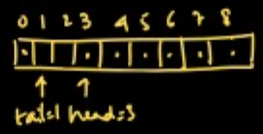
$0 \le r < n$

$-1/3$

$-1/3 = q$

$\boxed{-1 = 3(0) + (-1)}$ Java - allows negative remainders

$\boxed{-1 = 3(-1) + (2)}$ Math - remainder is non-negative



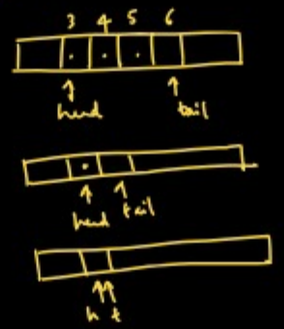
$sz = \underbrace{[(1-3)+9]}_{0 \le \ < 9} \% 9$

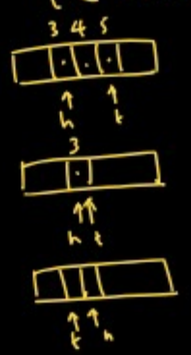
$sz = t - h = \underbrace{[(7-3)+9]}_{} \% 9$

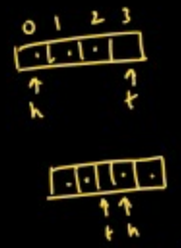
$7 - 3 = 4$

$[(7-3)+9] \% 9 = 4$

① tail → +1 of last element

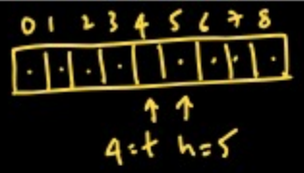


② 



$h == t+1$

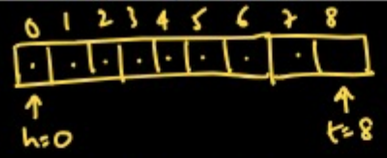
$(h-t)-1 == 0$



$h := t+1$

$h-t == 1$

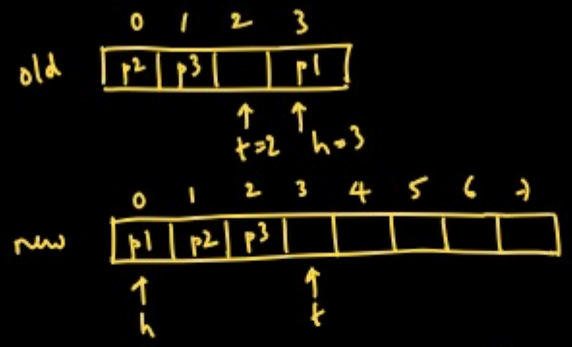
$(h-t)-1 == 0$



$[((h-t)-1) = -8-1 = -9 = 0 + 9] \% 9$

$$\boxed{(((h-t)-1)+9)\%9 == 0}$$

$$\boxed{size()+1 == 9}$$



$i = 0$ $\quad$ $new[i] = old[i + head] \Rightarrow new[0] = old[3]$

$i = 1$ $\quad$ $\boxed{new[i] = old[(i + head) \% 4]} \Rightarrow new[1] = old[4\%4] = old[0]$

$i = 2$ $\quad$ $new[2] = old[5\%4] = old[1]$

$i < size() = 3$

Insert:

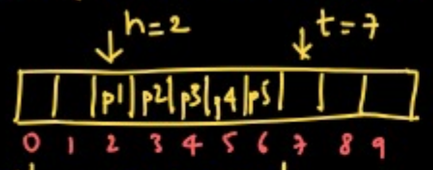
- index out of bounds
- size not being updated
- head / tail not being updated
- when list is empty head & tail would be null

pseudo code

1. handle invalid index
1.5. handle empty list → update tail & head
2. handle case where index is 0
   → update head
   → exit
3. handle case where index is size
   → update tail
4. handle general case

→ 1. check index is 0
   → delete head (update head)
2. check index is size-1
   → delete tail (update tail)
3. check if list has one element
   → update tail & head

size = ((t-h) + arr.length) % arr.length



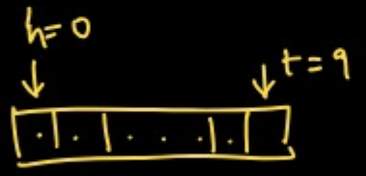
arr.length = 10

Case 1: No wrap around

size = (t-h) = 7-2 = 5

$\boxed{0 \le t-h < 10}$

↓size



$\Rightarrow \boxed{(t-h) \% 10 = (t-h)}$

t-h==0 ⇒ 0 % 10 = 0

t-h==1 ⇒ 1 % 10 = 1

⋮

t-h==9 ⇒ 9 % 10 = 9

size = (t-h) = (t-h) % 10 = [(t-h)+10] % 10

h=2
t=7

sz = 5

sz = ((7-2)+10) % 10
= 5

57 % 10 = 7

57 = 5(10) + 7
= 10k + 7

$$\begin{array}{r} 10k+7 \\ +\ 10 \\ \hline 10(k+1)+7 = \boxed{10(k_2)+7} \end{array}$$

n - dividend

m - divisor

q - quotient

r - remainder

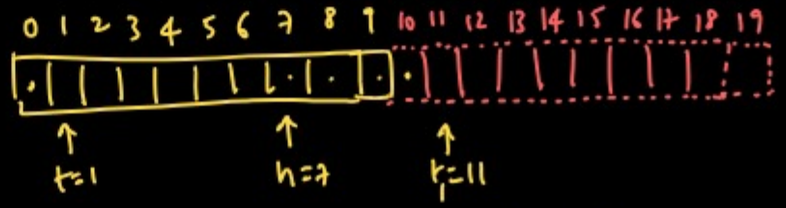
$\boxed{n = m.q + r}$

$0 \le r < m$

n % m = r

$\frac{57}{10} = 5\frac{7}{10}$

Case 2: Wraparound



size = $t_1 - h$

$t_1 = t + 10$

= (t+10) - h

$\boxed{\text{size} = (t-h) + 10}$ → established

0 ≤ size < 10

$\boxed{0 \le (t-h) + 10 < 10}$ → established

$\Rightarrow \boxed{(t-h)+10 = [(t-h)+10] \% 10}$ → established

size

size

if $0 \le k < 10$ then $k \% 10 = k$

Modus Ponens

- "If P then Q" is true
- P is true
- $\Rightarrow$ Q is true

"if x is even, then x^2 is even" (proved)

$x = 6$ = even

x^2 is even

(x is even) & (if x is even, then x^2 is even)

$\Rightarrow$ x^2 is even

- Applications of stacks

1. Checking if parantheses are balanced

[( )] [(( ))]	valid
[ )	invalid
([ )]	invalid
{ ([ )] }	invalid
[[(	invalid

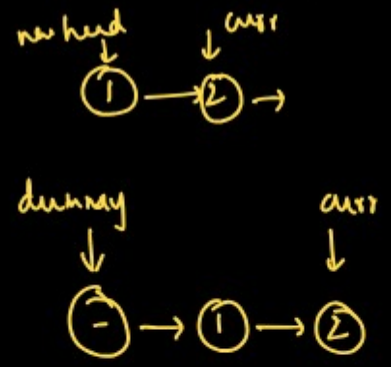
([

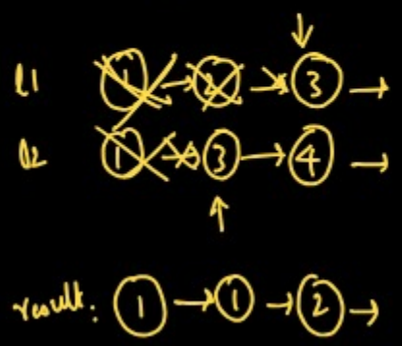
{ ([([ ])] ( ))

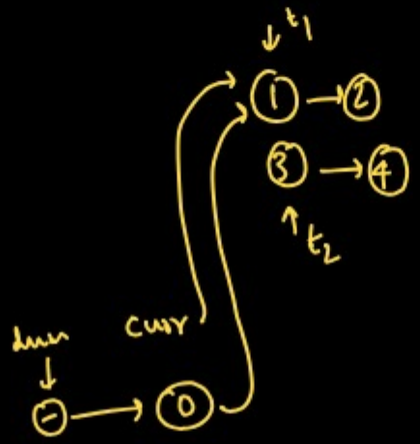
[ ]]
↑

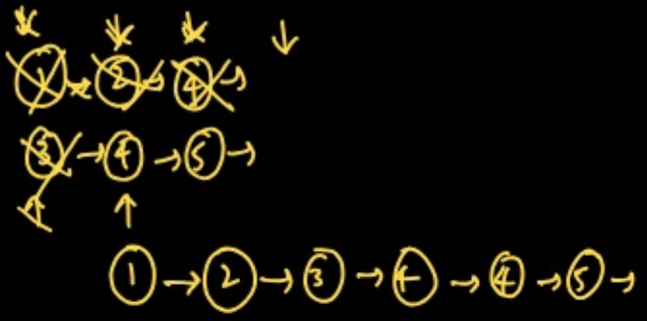
$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \qquad A^T = \begin{bmatrix} a & c \\ b & d \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}^T = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$









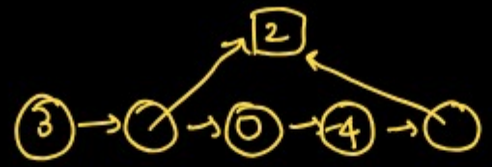
[1, 5] → from 1 to 5, both 1 & 5 included

[1, 5) → from 1 to 5 but 5 is excluded

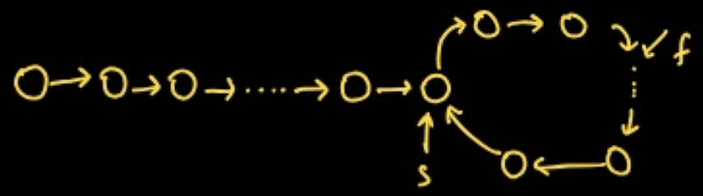
(1, 5)

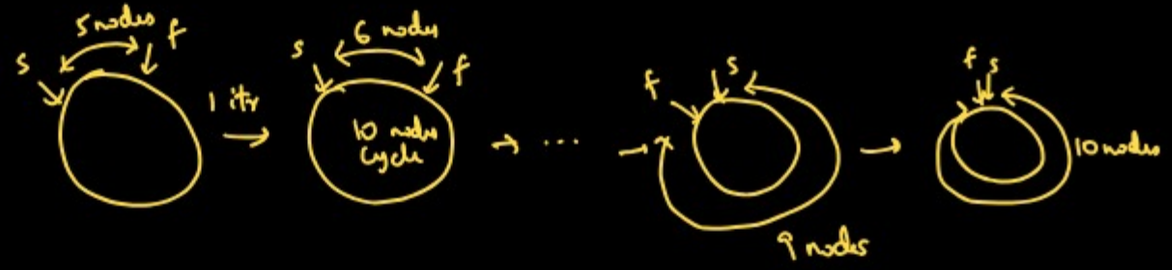
1.0001

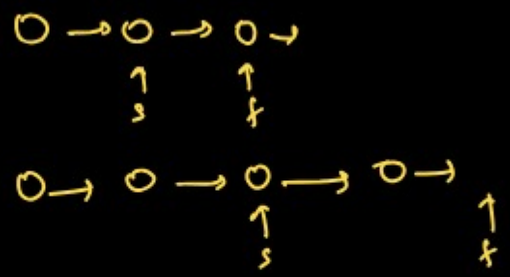
1.0



2 == 2







0 1 2 3 4 5 6

0
6
1
5
2
4
3

n1 → n2 →

dummy head

n0 → n3

tail

n0 → n1 → n2 → n3 → n4 →

split the list at the middle

l n0 → n1 → n2 →
r n3 → n4 →

reverse the second list

l n0 → n1 → n2 →
r n4 → n3 →

Merge 2 lists taking alternatively from l & r

n0 → n4 → n1 → n3 → n2 →

n0 n5 n1 n4 n2 n3

n0 → n1 → n2 → n3 → n4 → n5

n0 → n1 → n2 → n3 → n4 →

2 & 3
n1 → n2

n2 → n3

n0 → n1 →
n2 → n3 → n4

n0 → n1 → n2 →
n3 → n4 →

n0 → n1 →
n4 → n3 → n2 →

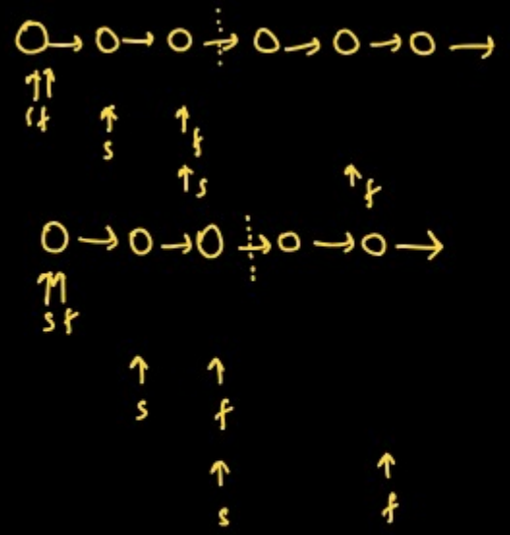
n0 → n1 → n2
n4 → n3 →

n0 → n4 → n1 → n3 ... → n2
(special case)

n0 → n4 → n1 → n3 → n2 →

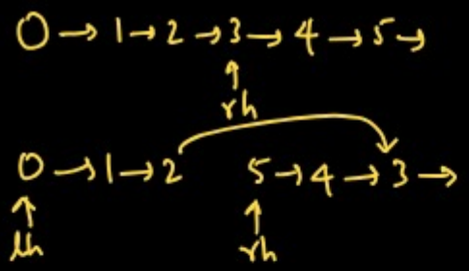
n0 → n1 →

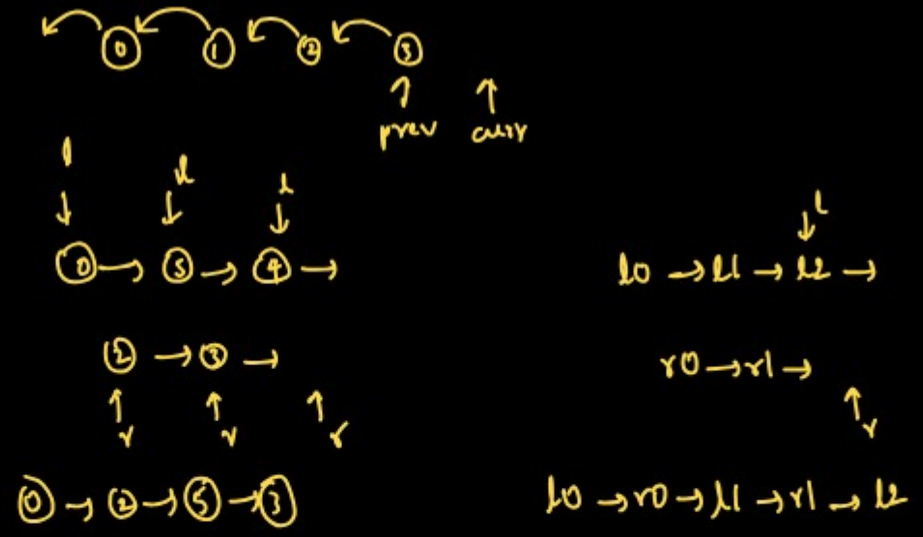
n0 → n1 →




```
while ( fast.next != null && fast.next.next != null )
    slow = slow.next
    fast = fast.next.next

// Break it up after slow
```





$$\begin{array}{r} 1\,1 \\ 1\,2\,3\,4 \\ +\quad 7\,8 \\ \hline 1\,3\,1\,2 \end{array}$$

$4 \to 3 \to 2 \to 1 \to$

$8 \to 7 \to$

32 bits $\to 2^{31} \to 2 \times 10^9$

$1 \to 2 \to 3 \to 4 \to$

$7 \to 8 \to$

$\leftarrow$

$1 \to 2 \to 3 \to 4 \to$

$0 \to 0 \to 7 \to 8 \to$

k=4 k=3 k=2 k=1

$0 \to 0 \to 0 \to 0 \to 0 \to 0 \to$

k=sz k=sz-1 k=sz-2 k=3 k=2 k=1

$0 \to 0 \to 0 \to \cdots \to 0 \to 0 \to 0 \to$

i=0 i=1 i=2 i=sz-3 i=sz-2 i=sz-1

k^{th} node from end

$\Rightarrow$ node with index $\boxed{i = sz - k}$ from the start

$\Rightarrow$ now we can delete the node at index $i = sz - k$

Step 1: Find the size

Step 2: Find the index of node to delete
$i = size - k$

Step 3: write a fn that deletes the node at index 'i'

$\overset{0}{0} \to \overset{1}{0} \to \overset{2}{0} \to \overset{3}{0} \to$